

Introdução ao Desenvolvimento de Jogos com Pygame

Arquitetura do Game Loop, Coordenadas 2D e Interatividade

Prof. Reginaldo Fernandes
Programação de Computadores / ADS

Instituto Federal do Ceará – Campus Tauá
Tecnologia em Análise e Desenvolvimento de Sistemas

9 de setembro de 2026

Sumário da Aula (Duração: 90 min)

- 1 Introdução e Filosofia do Pygame
- 2 A Anatomia do Game Loop
- 3 Coordenadas 2D e Cores RGB
- 4 Animação, FPS e Interatividade
- 5 Colisões com `pygame.Rect` e o Mini-Jogo
- 6 Laboratório Prático e Conclusão

O que é o Pygame?

- O **Pygame** é um conjunto de módulos em Python projetados para a criação de jogos bidimensionais (2D) e aplicações multimídia.
- Ele é construído sobre a biblioteca em C **SDL** (*Simple DirectMedia Layer*), combinando a agilidade do Python com a performance do hardware gráfico.

O que o Pygame gerencia?

- Criação de janelas gráficas.
- Renderização de formas e imagens.
- Captura de teclado, mouse e controles.
- Reprodução de sons e efeitos de áudio.
- Controle fino do relógio (FPS).

Instalação Simples

No terminal do laboratório:

```
pip install pygame-ce
```

pygame-ce (Community Edition) é a versão moderna mantida pela comunidade.

Programas Tradicionais vs. Jogos em Tempo Real

Aplicações Tradicionais (Web/CLI)

- Modelo **orientado a eventos passivo**.
- O programa fica **em repouso** aguardando o usuário digitar ou clicar.
- Se ninguém interagir, nada acontece na tela.

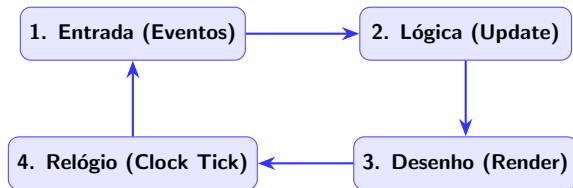
Jogos em Tempo Real

- Modelo de **Lazo Contínuo** (*Game Loop*).
- O mundo virtual continua acontecendo **60 vezes por segundo**.
- Inimigos andam, física se aplica e a tela redesenha, mesmo sem comandos.

Conclusão Didática: Um jogo é essencialmente uma animação interativa calculada em tempo real dentro de um laço de repetição infinito!

O Ciclo de Vida de um Jogo

Todo jogo de computador, do clássico Pong aos títulos AAA modernos, é executado dentro de quatro fases essenciais a cada quadro (*frame*):



- **1. Entrada:** Lê cliques do mouse, teclas pressionadas e fechamento da janela.
- **2. Lógica:** Atualiza posições, calcula colisões, pontuação e regras.
- **3. Desenho:** Limpa o buffer de vídeo e redesenha todos os elementos.
- **4. Relógio:** Aguarda o tempo necessário para travar a taxa de quadros (ex: 60 FPS).

O Código Mínimo Essencial (Esqueleto)

```
1 import pygame, sys
2
3 pygame.init()
4 tela = pygame.display.set_mode((800, 600))
5 pygame.display.set_caption("Meu Primeiro Jogo")
6 relógio = pygame.time.Clock()
7
8 executando = True
9 while executando:
10     # 1. Eventos
11     for evento in pygame.event.get():
12         if evento.type == pygame.QUIT:
13             executando = False
14
15     # 2. Lógica (vazio no momento)
16
17     # 3. Desenho
18     tela.fill((30, 30, 45)) # Limpa fundo com cinza escuro
19     pygame.display.flip()    # Exibe no monitor
20
21     # 4. Controle de FPS
22     relógio.tick(60)
23
24 pygame.quit()
25 sys.exit()
```

Por que o programa trava sem `pygame.event.get()`?

O Sistema Operacional e a Fila de Mensagens

- Os sistemas operacionais (Linux, Windows, macOS) enviam mensagens contínuas para as janelas abertas (foco, teclado, redimensionamento, botão de fechar).
- Se o jogo não consumir essa fila através de `pygame.event.get()`, o sistema assume que o processo entrou em congelamento (*loop infinito travado*) e exibe a mensagem de “Não Respondendo”.

O que faz `pygame.QUIT`?

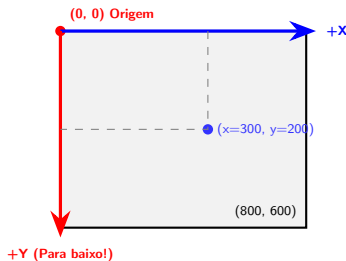
- Representa o clique no botão vermelho de fechar janela ‘X’.
- Permite sair suavemente do laço `while`.

O que faz `display.flip()`?

- Implementa o **Double Buffering**: tudo é desenhado na memória oculta e depois trocado de uma só vez para a tela visível, evitando cintilação (*flickering*).

O Plano Cartesiano na Computação Gráfica

Atenção: As coordenadas de tela na computação gráfica são diferentes do plano cartesiano que aprendemos na matemática tradicional!



Regras Fundamentais

- O ponto **(0, 0)** fica no canto **superior esquerdo** (*Top-Left*).
- O eixo **X** cresce para a **DIREITA**.
- O eixo **Y** cresce para **BAIXO**!
- Logo, para descer um objeto na tela, **somamos** em **Y**; para subir, **subtraímos** de **Y**.

O Modelo de Cores Aditivas: RGB

As cores no Pygame são representadas por tuplas de 3 números inteiros variando entre 0 e 255: **(Red, Green, Blue)**.

Cor	Tupla (R, G, B)	Significado
Preto	(0, 0, 0)	Ausência total de luz
Branco	(255, 255, 255)	Intensidade máxima de todos os canais
Vermelho	(255, 0, 0)	Apenas canal R ativo
Verde	(0, 255, 0)	Apenas canal G ativo
Azul	(0, 0, 255)	Apenas canal B ativo
Amarelo	(255, 255, 0)	Mistura de Vermelho + Verde
Ciano	(0, 255, 255)	Mistura de Verde + Azul

Dica de Código

Defina cores como constantes no topo do arquivo com nomes em **MAIÚSCULAS** para deixar seu código legível: **AZUL = (50, 150, 250)**.

Desenhando Primitivos com `pygame.draw`

O módulo `pygame.draw` permite criar formas na tela sem precisar de imagens externas:

```
1 # 1. Retangulo: (tela, cor, (x, y, largura, altura), espessura)
2 pygame.draw.rect(tela, (0, 120, 255), (100, 150, 80, 50))
3
4 # 2. Circulo: (tela, cor, (centro_x, centro_y), raio, espessura)
5 pygame.draw.circle(tela, (255, 50, 50), (400, 300), 40)
6
7 # 3. Linha: (tela, cor, (inicio_x, inicio_y), (fim_x, fim_y), espessura)
8 pygame.draw.line(tela, (255, 255, 255), (50, 50), (200, 50), 3)
9
10 # 4. Poligono (ex: triangulo): (tela, cor, [(x1,y1), (x2,y2), ...])
11 pygame.draw.polygon(tela, (255, 220, 0), [(300, 80), (250, 180), (350, 180)])
```

- Se o argumento de espessura (`width`) for omitido ou for 0, a forma é **totalmente preenchida**.
- Se for maior que 0, é desenhada apenas a **borda**.

Como Criar a Ilusão de Movimento?

A Lógica da Física:

- Uma animação consiste em mudar levemente as coordenadas (x, y) de um objeto a cada passagem pelo laço principal.
- Equação básica da velocidade constante:

$$x \leftarrow x + v_x$$

$$y \leftarrow y + v_y$$

O Erro Clássico do Rastro

Se você esquecer de chamar `tela.fill(COR)` antes de desenhar o novo quadro, a tela não será limpa!

O objeto desenhado no quadro anterior continuará lá, criando um rastro borrado infinito.

O Papel Crucial de `pygame.time.Clock()`

O que acontece sem o controle de FPS?

- O computador rodará o `while` na velocidade máxima que o processador aguentar (milhares de ciclos por segundo).
- O jogo consumirá **100% de CPU**, e rodará ultra rápido em máquinas potentes e lento em computadores antigos!

Com `relógio.tick(60)`

- O Pygame calcula quanto tempo o quadro demorou e coloca a CPU para dormir (*sleep*) pelo tempo restante.
- Garante que o jogo rode a exatos **60 FPS** em qualquer computador.

Fórmula do Tempo de Quadro:

$$\Delta t = \frac{1000 \text{ ms}}{60 \text{ FPS}} \approx 16,6 \text{ ms por quadro}$$

Movimentação Interativa pelo Teclado

Para movimentar personagens de forma contínua e suave, utilizamos `pygame.key.get_pressed()`:

```
1 # Captura o estado contínuo de todas as teclas
2 teclas = pygame.key.get_pressed()
3
4 # Movimento Horizontal (Setas ou A / D)
5 if teclas[pygame.K_LEFT] or teclas[pygame.K_a]:
6     jogador_x -= velocidade
7 if teclas[pygame.K_RIGHT] or teclas[pygame.K_d]:
8     jogador_x += velocidade
9
10 # Movimento Vertical (Setas ou W / S)
11 if teclas[pygame.K_UP] or teclas[pygame.K_w]:
12     jogador_y -= velocidade
13 if teclas[pygame.K_DOWN] or teclas[pygame.K_s]:
14     jogador_y += velocidade
```

Diferença Fundamental:

- `pygame.KEYDOWN` (na fila de eventos): Dispara **uma única vez** quando a tecla desce (ótimo para tiro ou pulo).
- `pygame.key.get_pressed()`: Responde **enquanto a tecla estiver pressionada** (ótimo para corrida).

O Poder da Classe `pygame.Rect`

O `pygame.Rect` é a estrutura mais versátil do Pygame para representar a posição e as dimensões de objetos 2D.

Criação

```
r = pygame.Rect(x, y, larg, alt)
```

Propriedades automáticas:

- `r.x`, `r.y`
- `r.width`, `r.height`
- `r.top`, `r.bottom`
- `r.left`, `r.right`
- `r.center`, `r.centerx`, `r.centery`

Bloqueio de Bordas Fácil

- `if r.left < 0: r.left = 0`
- `if r.right > 800: r.right = 800`
- `if r.top < 0: r.top = 0`
- `if r.bottom > 600: r.bottom = 600`

Não é necessário recalcular largura ou altura manualmente!

Detecção de Colisão: `colliderect()`

Para saber se dois objetos se tocam no espaço 2D, usamos a colisão por caixas alinhadas aos eixos (*AABB - Axis-Aligned Bounding Box*):

```
1 jogador = pygame.Rect(100, 100, 40, 40)
2 moeda    = pygame.Rect(300, 200, 20, 20)
3
4 # O metodo colliderect devolve True se houver sobreposicao!
5 if jogador.colliderect(moeda):
6     pontos += 1
7     # Reposiciona a moeda em outro local aleatorio
8     moeda.x = random.randint(50, 750)
9     moeda.y = random.randint(50, 550)
```

Princípio Matemático AABB

Dois retângulos colidem se, e somente se, houver sobreposição simultânea em ambos os eixos X e Y :

$$(A_{\text{left}} < B_{\text{right}}) \wedge (A_{\text{right}} > B_{\text{left}}) \wedge (A_{\text{top}} < B_{\text{bottom}}) \wedge (A_{\text{bottom}} > B_{\text{top}})$$

Escrevendo Textos na Tela (HUD de Pontuação)

Para desenhar pontuações, vidas e mensagens na tela, seguimos três passos:

1 Carregar a Fonte:

```
1 fonte = pygame.font.SysFont("Arial", 24, bold=True)
```

2 Renderizar o Texto em uma Surface:

```
1 # Argumentos: (texto, antialiasing_suave, cor_RGB)
2 img_texto = fonte.render(f"Pontos: {pontos}", True, (255, 255, 255))
```

3 Transferir a Imagem para a Tela (blit):

```
1 # Posiciona a imagem de texto nas coordenadas (x=20, y=20)
2 tela.blit(img_texto, (20, 20))
```

Atenção

A palavra **blit** vem de *Block Image Transfer* (copiar blocos de pixels de uma superfície para outra).

Resumo dos Pilares Aprendidos

Nesta aula introdutória de 90 minutos, dominamos os quatro alicerces fundamentais da computação gráfica 2D:

- **Game Loop:** A espinha dorsal contínua que processa entradas, física e renderização.
- **Plano Cartesiano 2D:** Origem $(0,0)$ no topo esquerdo e coordenada Y invertida para baixo.
- **Controle de Tempo:** Uso de `Clock.tick(60)` para garantir taxa estável em qualquer hardware.
- **Colisões com Rect:** Otimização e praticidade através do método `collidirect()`.

Boas Práticas de Engenharia de Jogos

- Sempre limpe a tela com `tela.fill()` antes de desenhar.
- Trate o evento `pygame.QUIT` para não congelar o sistema operacional.
- Separe a lógica de atualização da renderização visual.

Desafio 1: A Bola Multicolorida (Nível Fácil)

- Execute o arquivo `desafio1_bola_quicando.py`.
- Faça a bola quicar continuamente nas 4 paredes da janela.
- **Bônus:** A cada batida, sorteie uma nova cor aleatória com `random.randint(50, 255)`.

Desafio 2: Esquiva de Meteoros (Nível Intermediário)

- Execute o arquivo `desafio2_esquiva_obstaculo.py`.
- Mova seu personagem na horizontal para desviar de blocos caindo do topo da tela.
- Utilize `colliderect()` para descontar vidas do jogador se for atingido!

O que exploraremos nas próximas aulas?

- **Sprites e Grupos:** Uso da classe `pygame.sprite.Sprite` e `pygame.sprite.Group` para gerenciar dezenas de entidades com POO.
- **Carregamento de Imagens:** Substituir formas geométricas por Spritesheets e animações de personagens.
- **Sistemas de Partículas e Câmera:** Efeitos visuais dinâmicos e mundos maiores que a tela.

Obrigado!

IFCE Campus Tauá – Curso de Análise e Desenvolvimento de Sistemas